COMPUTING METHODS IN HIGH ENERGY PHYSICS

S. Lehti

Helsinki Institute of Physics

Spring 2026

Detector simulations

Introduction

In high energy physics, the method to simulate experimental data is done in two stages, event generation and detector simulation. Event generators describe the particle reactions and produce the momentum vectors of the generated particles. The output of an event generator is used as input for a detector simulation program.

The response of a detector to the passage of the scattered particles involves random processes such as ionization, multiple Coulomb scattering etc. Therefore the transport of particles through an experimental setup can be effectively implemented using the Monte Carlo technique. Programming packages, such as GEANT can be used to describe complicated detector configurations. Many GEANT based simulation packages has been developed to offer central facilities for Monte Carlo studies for different experiments.

GEANT4

Geant4 is a free software package composed of tools which can be used to accurately simulate the passage of particles through matter. The simulation processes included in the toolkit are

- geometry of the system
- materials involved
- particles

Detector simulations

- generation of events
- tracking through matter and EM fields
- response of sensitive detector components
- generation of event data
- storage of events and tracks
- visualization
- analysis of data

Users may construct stand-alone applications, or applications built upon another object-oriented framework.

The Geant4 physics models handle the interactions of particles with matter across a very wide energy range. It incorporates a large part of all that is known about particle interactions.

The Geant4 web page url is http://geant4.cern.ch

For the detector simulation one needs a detector description. The detector definition requires the representation of its geometrical elements, their materials and electronics properties. The geometrical representation of detector elements focuses on the definition of solid models and their spatial position.

Geant4 is capable of describing and propagating in variety of electromagnetic fields. Magnetic

Detector simulations

fields, electric fields and electromagnetic uniform and non-uniform fields can be specified. In order to propagate a particle inside a field, the equation of motion of the particle in the field is integrated in the toolkit.

A physical interaction of a particle in the sensitive region of a detector is called a hit. Hits simulate the physical signal in the detector element produced by the particle-matter interaction.

The physical signal in the sensitive detector element, the detector response, is transformed with the help of electronics into a digital signal, which can be stored on tape or hard drive. The simulation of this step is called digitization, and the digitized data, the raw data ("digis"). Typical usages of the digitizer are

- Simulation of ADC and/or TDC
- Simulation of readout scheme
- Generation of raw data
- Simulation of trigger logics
- Simulation of pile-up

At high luminosities several separate events are likely to occur at the same bunch crossing. These so called minimum bias events or pile-up add a non-negligible level of background to the response of the detector system. In the tracker the pile-up events generate a number of hits which can affect the track finding efficiency and fit accuracy.

Detector simulations

In the calorimeters these events add pile-up noise affecting calorimeter and missing energy resolution.

Physics processes describe how particles interact with the material. Seven major categories are provided by Geant4:

- electromagnetic
- hadronic
- decay
- photolepton-hadron
- optical
- parametrization
- transportation

Electromagnetic physics processes include

- Photon processes, like Compton scattering, gamma conversion, photo-electric effect and muon pair production
- Electron/positron processes, like ionization and delta rays, bremsstrahlung and positron annihilation
- Muon processes, like ionization and delta rays, bremsstrahlung and e+e- pair production
- Hadron/ion processes, like ionization
- Multiple scattering
- Low energy electromagnetic processes, like Rayleigh scattering

Detector simulations

Hadronic interactions include

- lepton-hadron interactions
- photonuclear and electronuclear reactions
- nucleus-nucleus reactions
- elastic scattering
- interactions of stopping particles
- nuclear cascades
- fission, evaporation, break-up models
- low energy neutron interactions
- radioactive decay

The optical effects include processes like Cerenkov radiation, scintillation, wavelenght shifting and absorption.

The algorithm

Probability of having an interaction within L due to physics process i is

$$p_i(L) = 1 - exp(-L/\lambda_i) \quad (1)$$

(λ_i= mean free path). The Monte Carlo algoritm:

1. Set properties for incident particle (momentum..)
2. Get values for λ_i for all relevant processes
3. Sample the free path length L from the distributions of all participating processes
4. Select the smallest path length
5. Move the particle by this step
6. Simulate the interaction
7. if particle still exists, goto 1

Detector simulations

Free path lengths sampling produces steps.

A step is always created at the material boundaries.

Physics processes limit the step size according to the sample's free path length.

E.g. zick-zack path due to elastic Coulomb scattering.

E.g. Energy loss and possible creation of other particles

The physics process is a microscopic description of the particle interaction with another particle of the material or the external field, accoring to quantum theory.

The Detector Description must be as realistic as possible for the physics to be studied. Particles tracked through the detector must see and feel a realistic environment in order to deliver realistic simulation results.

The Detector Description need lots of experience and expertise, one has to know which details of the detector are essential and which can be safely ignored. Often the detector cannot be modelled 1:1, e.g. the geometry must fit into the computer memory..

Usage of Geant4

Since the contents of the program varies according to the needs of a

Detector simulations

given simulation, the Geant4 toolkit does not provide the main program. The main must be written by the user. Examples of simple main programs are, however, provided. The main() method is implemented by two toolkit classes

- G4RunManager
- G4UImanager

The first thing main() must do is to create an instance of the G4RunManager class. It controls the flow of the program and manages the event loops within a run. The run manager is responsible for initialization procedures, and it must be given all the information necessary to build and run the simulation. This includes

- detector description
- physics processes
- how the primary particles are produced
- any additional requirements

Example 1:

```
G4RunManager* runManager = new
G4RunManager;
runManager->SetUserInitialization(new
      ExN01DetectorConstruction);
runManager->SetUserInitialization(new
      ExN01PhysicsList);

runManager->SetUserAction(new
      ExN01PrimaryGeneratorAction);

runManager->initialize();
```

Detector simulations

```
G4UImanager* UI =
G4UImanager::GetUIpointer();
UI->ApplyCommand("/run/verbose
1");
UI-
>ApplyCommand("/event/verbose
1");
UI-
>ApplyCommand("/tracking/verbose
1");

int numberOfEvents = 3;
runManager-
>BeamOn(numberOfEvents);

delete runManager;
```

The detector description specifies the detector geometry, used materials, defines sensitive regions and readout schemes.

The geometry defines the size and shape, like Boxes, Tubes and their sections, Cones and their sections, Spheres, Wedges, and Toruses.

```
G4Tubs(const G4String& pName,
            G4double pRMin,
            G4double pRMax,
            G4double pDz,
            G4double pSPhi,
            G4double pDPhi)
```

```
G4Material* Al = new
G4Material("Aluminum",
z= 13., a= 26.98*g/mole, density=
2.7*g/cm3);
```

Every quantity such as mass, energy, density,.. is represented by

Detector simulations

multiplying the numerical value with the unit. For reading returned quantities, the numerical value should be divided by the unit:
double speed = getSpeed()/(km/h);

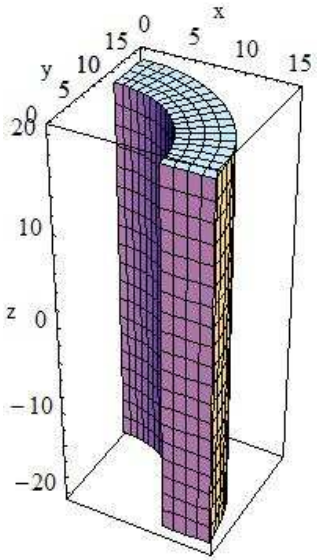

Figure: pRMin = 10, pRMax = 15, pDz = 20 pSPhi = 0*Degree, pDPhi = 90*Degree

The Logical Volume manages the information associated with detector elements represented by a given Solid and Material, independently from its physical position in the detector.

Physical volumes represent the spatial positioning of the volumes describing the detector elements. Several techniques can be used. They range from the simple placement of a single copy to the repeated positioning using either a simple linear formula or a user specified function.

SensitiveDetector represents the parts of the detector that will do the actual measurement. Sensitivity is handled via logical volumes, and the

Detector simulations

principal mandate of a sensitive detector is the construction of hit objects using information from steps along a particle track.

Readout geometry is a virtual, parallel geometry for obtaining the channel number. It associates ReadoutGeometry object with a sensitive detector.

The primary particle production can be done with particle guns, or one can use interface to external event generators. For Geant4 it was decided not to link any FORTRAN code in the program. Instead, Geant4 provides an ASCII file interface for such event generators.

In the example 1 the PhysicsList defines a geantino, and assigns transportation process. The PrimaryGeneratorAction constructs a particle gun and generates events by shooting geantinos with the gun. Geantino is a non-physical test particle which simulates particle transportation, it does not interact with material.

Geant4 is designed to transport long lived particles along macroscopic distances. We have to tell Geant4 the initial conditions of the particles we want to have simulated. The short lived particles like Z will not be tracked.

User has to instantiate the Primary Vertex.

Detector simulations

The G4UImanager is used for giving optional UI (or GUI) commands via the member function ApplyCommand. The commands are listed in the user guide (for app.dev) section 7.1. The command

/run/verbose 1

is selected to display main topics, the command

/event/verbose 1

to give stacking information, and command

/tracking/verbose 1

gives minimum information of each Step. These all have default values 0, which give silent operation.

Summary:

The various stages during a simulation run are

- Initialization
  - detector description
  - physics processes selection and configuration
- Run
  - contains several Events under the same conditions
- Event
  - generation of primary particles
  - tracking of all particles

We do not go deeper into the usage of Geant4, as normally one does not have to use Geant directly, but via an auxiliary program where the geometry etc is already defined and the end user only provides the primary event for the detector

Detector simulations

simulation to get a hit collection. Very often experiments do not use Geant4 digitization, but implement their own digitization machineries, like in CMS.

For further reading, Geant4 provides extensive documentation like

- User's guide for application developer (basic usage of Geant4)
- Physics reference manual
- Software reference manual

and most of all working examples for beginners, extended examples and examples for advanced users.

Detector simulations

Dataset creation for full simulation

Instead of using standalone GEANT simulations, the HEP experiments often create their own applications and simulation frameworks. The full GEANT simulation is used, but via a customized interface, different for each experiment as the code is written from scratch by the physicists participating the experiment. Therefore an average user does not have to worry about e.g. a correct detector geometry description, as the geometry is modelled by experts and included in the simulation framework.

CMS is no exception, it has its own programs written for end users. The first full simulation program CMSIM was written in Fortran, and it used Fortran based GEANT3 for simulating the particle-matter interactions. Later CMSIM was replaced with OO software ORCA, OSCAR and CMSSW, and GEANT3 was switched to GEANT4. CMSSW uses an OO framework, any Fortran code still living is hidden by C++ interfaces to the Fortran programs. We use CMS here as an example to get the general idea, how the full simulation works in practice.

CMSSW

The dataset production for CMSSW simulations can be done in steps. The steps for a full simulation study are

Detector simulations

- Event generation
- Hits
- Digis
- High level object reconstruction
- Analysis

Event generation is done using the Monte Carlo event generators available. The most common event generators are included in the program as libraries, and the user can select the event generator, and what kind of events are to be simulated. The output is a root file containing the data collections user has selected.

Framework

The CMSSW framework implements a software model wherein there is one executable, called cmsRun, and many plug-in modules which run algorithms. The same executable is used for both detector simulation and for creating the Monte Carlo data.

The CMSSW executable, cmsRun, is configured at run time by the user's job-specific configuration file. This file tells cmsRun which data to use, which modules to run, which parameter settings to use for each module, and in what order to run the modules. Required modules are dynamically loaded at the beginning of the job. The configuration file language is Python.

```
$ cmsRun myPythonCfgFile.py
```

Detector simulations

The CMS Event Data Model (EDM) is centered around the concept of an Event as a C++ object container for all RAW and reconstructed data pertaining to a physics event. During processing, data are passed from one module to the next via the Event, and are accessed only through the Event. All objects in the Event may be individually or collectively stored in ROOT files, and are thus directly browsable in ROOT.

The CMS code can be found at GitHub
`https://github.com/cms-sw/cmssw`

and the reference manual at
`https://cmsdoxygen.web.cern.ch/cmsdoxygen/`

Environment setup

On kale.grid, the environment needed for CMSSW is set by

```
setenv VO_CMS_SW_DIR
/cvmfs/cms.cern.ch
source
$VO_CMS_SW_DIR/cmsset_default.csh
setenv SCRAM_ARCH
slc7_amd64_gcc830
```

Dataset creation

To create a new dataset one first needs to create a working area. This is done with an auxiliary program called SCRAM (Source Configuration, Release, And Management). With command

Detector simulations

"scram list" you can see all the software and versions available via scram.

```
$ scram project CMSSW CMSSW_10_6_22
```

```
$ cd CMSSW_10_6_22/src/
```

set the runtime environment:

```
$ cmsenv
```

The source code (Gen-fragment) is in the git repo,

```
git cms-addpkg Configuration/Generator
```

Compile:

```
$ scram b -j 16
```

Next, using cmsDriver.py, create a configuration file for the generation of the events in which Higgs particle decays to two Z^0 bosons which in turn decay to four leptons:

```
cmsDriver.py
Configuration/Generator/python/ \
H200ZZ4L_13TeV_TuneCUETP8M1_cfi.py \
-s GEN --conditions auto:run2_mc_GRun \
--datatier GEN-SIM \
--eventcontent RAWSIM --no_exec
```

This generates the events. For the detector simulation, change '-s GEN' to '-s GEN,SIM'.

The generated config file is the input for running CMSSW (cmsRun). The number of events to be simulated is controlled with the line

```
process.maxEvents = cms.untracked.PSet(
    input = cms.untracked.int32(1))
```

Pile-up is selected by including a pile-up config file, in this case no pile-up

```
process.load(``SimGeneral.MixingModule.
mixNoPU_cfi'')
```

The output file name is selected with PoolOutputModule. In PoolOutputModule one can also select the saved collections, e.g. if one wants only the generator level

Detector simulations

information, use drop:

```
process.FEVT =
cms.OutPutModule(‘‘PoolOutputModule’’,
  cms.PSet(outputCommands=cms.untracked.
  vstring(’’drop *’’,
        ’’keep *_generator_*_*’’))

  fileName = cms.untracked.string(...)
```

The wildcard * in the drop and keep commands replace only whole strings, not substrings

`"keep *_TriggerResults_*_*"`, Allowed

`"keep *_Trigger*_*_*"`, Won't work.

Let's next browse the root file content. Open the root file and start the browser
$ root H200..._GEN.root
root [1] new TBrowser
double click H200..._GEN.root
double click Events;1

Now you have all the collections listed. The collection names are formed from namespace+Class name _ label _ instance _ process name

Example: For MC info the namespace is edm, class name HepMCProduct, label generator, no instance and produced with process GEN, the name of the process in the example thus making edmHepMCProduct_generator__GEN
How is the data then accessed? One needs to write an analysis module (class), by inheriting an edm::EDAnalyzer. In the analysis code the MC generator level information in edmHepMCProduct_generator__GEN

Detector simulations

is accessed as follows:
The class name was edm::HepMCProduct, and label "generator".

```
Handle<edm::HepMCProduct> mcEvtHandle;
iEvent.getByToken(mcToken,mcEvtHandle);
if(mcEvtHandle.isValid()){
   const HepMC::GenEvent* ev =
      mcEvHandle->GetEvent();
   HepMC::GenEvent::
   particle_const_iterator i;
   for(i = ev->particles_begin();...
   }
}
```

Let's next modify the example Gen-fragment to produce SM Higgs bosons with one of the important SM Higgs production processes, the Vector Boson Fusion (VBF). The Higgs boson decays mainly to $\mathrm{b\bar{b}}$ pairs, but the b jet final state is difficult to extract from the QCD multijet background. The next most probable final state is $\mathrm{H} \to \tau\tau$, which is experimentally easier. The PYTHIA manual tells the correct PYTHIA switches for these kind of events, section Process selection – Higgs. The vector boson fusion is already switched on in the example datacard (processes HiggsSM:ff2Hff(t:ZZ) and HiggsSM:ff2Hff(t:WW)) so let's keep them and comment out the loop mediated process. The decays should now include only decays H$\to \tau\tau$. Since the SM Higgs KF code is 25 and tau KF code is 15, we change the config as follows

```
'25:onMode = off',
'25:onIfMatch = 15 15',
```

The Higgs decay final states can be

Detector simulations

listed by running stand-alone PYTHIA calling pythia.particleData.list(), as described in Lecture 8. The example Gen-fragment looks now like this:

```
generator =
cms.EDFilter("Pythia8GeneratorFilter",
  PythiaParameters = cms.PSet(
    parameterSets = cms.vstring(
      'pythia8CommonSettings',
      'pythia8CUEP8M1Settings',
      'processParameters'),
    processParameters = cms.vstring(
      'HiggsSM:ff2Hff(t:WW) = on',
      'HiggsSM:ff2Hff(t:ZZ) = on ',
      '25:m0 = 200',
      '25:onMode = off',
      '25:onIfMatch = 15 15'),
```

The process parameters define the hard interaction, but the definitions for the underlying event are needed, too. They are added by pythia8CommonSettings and (for the tune CUEP8M1, there are also other tunes available) pythia8CUEP8M1Settings. The CMSSW configuration files for event generation with different event generators are available in the git repository. There is also a link from the MCM page (passwd protected) to each dataset and Gen-fragment

`https://cms-pdmv.cern.ch/mcm`

This allows one to make a semi-private production: private event generation with official datacards, and it provides working examples which can serve as a starting point for creating new Gen-fragments.

Detector simulations

It is also possible to create LHE files separately, with stand-alone tools, and then use those LHE files to feed official CMS dataset production. If you need to do this, contact your physics analysis group or physics object group generator contact person for further instructions.

Analysis example

For CMSSW analysis one needs data, and an analysis program. Let's assume we have a dataset available containing electrons as in the dataset generation example. The analysis program (module) needs to be written. Since we are now interested in electrons, let's start from an Electron Analysis example, found under RecoEgamma/Examples

```
$ scram p CMSSW CMSSW_10_6_22
$ cd CMSSW_10_6_22/src
$ cmsenv
$ git cms-addpkg
RecoEgamma/Examples
$ cd RecoEgamma/Examples
$ scram b -j 16
```

We select the example GsfElectronMCAnalyzer. The source code is located in plugins/. First let's try to run the analyzer without modifications.

```
$ cd test
```

Try grepping to find the config file for module name GsfElectronMCAnalyzer. The file is GsfElectronsMCAnalyzer_cfg.py. Edit the file to contain the correct

Detector simulations

input (assuming the produced root file is moved to $PWD),

```
process.source =
cms.source("PoolSource",
 fileNames =
cms.untracked.vstring (
   "file:H200_GEN.root"
 )
)
```

The prefix 'file:' is needed when accessing local root files. If the file is read from CASTOR, prefix 'rfio:' is needed. If no prefix is specified, the default location for datasets /store.. is used.

Let's select `maxEvents = 2`, and run the program:

```
$ cmsRun
GsfElectronsMCAnalyzer_cfg.py
```

Since the input file contains only generator level information, the example is not expected to work without commenting out the reco-object code. Let's leave only the MC particle loop, and modify the code to print the MC electron pt, eta and phi. Edit file plugins/GsfElectronMCAnalyzer.cc member function GsfElectronMCAnalyzer::analyze

One can see there a loop over MC particles (GenParticleCollection is used here), where the print should be added. The reco::GenParticleCollection contains the same information as the edm::HepMCProduct (standard HepMC event record), but the

Detector simulations

former is recommended as the code will work with all data tiers, like AOD, from which the edm::HepMCProduct is dropped.
Recompile and rerun

```
$ scram b
$ cmsRun
GsfElectronsMCAnalyzer_cfg.py
```

More about reco::GenParticle methods can be found in the CMSSW reference manual.